

Divide and Conquer Algorithms

CMSC 27200: Theory of Algorithms

January 22 - January 29, 2025

Corresponding section(s) of Kleinberg-Tardos: 5.1,5.2,5.3,5.4,5.6

1 Overview

Divide and conquer algorithms solve problems by splitting them into smaller subproblems and then cleverly combining the solutions for the subproblems into a solution for the entire problem.

2 Recursive Algorithms and Recurrence Relations

By their nature, divide and conquer algorithms are usually recursive algorithms which call themselves. To analyze the performance of recursive algorithms, we let $T(n)$ be the time the algorithm takes when the input has size n and write down a recurrence relation for $T(n)$. We will then try and solve this recurrence relation.

Example 2.1 (Binary search). *The binary search algorithm checks if an element x is in a sorted array A as follows:*

Input to the recursive algorithm: The recursive algorithm requires indices i and j such that if x is in A , it must be located at some index $i' \in [i, j]$.

Recursive algorithm: If $j < i$ then x cannot be in A and we reject. Otherwise, take $k = \lfloor \frac{i+j}{2} \rfloor$. We now have the following cases:

- (a) If $x = a_k$ then we have found x .*
- (b) If $x > a_k$ then we use binary search with $i' = k + 1$ and $j' = j$.*
- (c) If $x < a_k$ then we use binary search with $i' = i$ and $j' = k - 1$.*

To check if an element x is in A , we initially call the recursive algorithm on A with $i = 0$ and $j = n - 1$.

Up to a constant factor, the recurrence relation for binary search is $T(n) = T(\frac{n}{2}) + 1$. As discussed in the notes on solving recurrence relations, this implies that $T(n)$ is $O(\log n)$.

Example 2.2 (Mergesort). *The mergesort algorithm sorts an array of n elements as follows: If $n = 1$, the array is already sorted. Otherwise, we take the following steps:*

1. Mergesort elements 0 through $\lfloor \frac{n}{2} \rfloor - 1$ of the array.
2. Mergesort elements $\lfloor \frac{n}{2} \rfloor$ through $n - 1$ of the array.
3. If A and B are two sorted arrays of lengths n_1 and n_2 and then we can merge these arrays into a sorted array C of length $n_1 + n_2$ as follows:

Initialization: Start with $i = 0$ and $j = 0$.

Iterative step: If $j \geq n_2$ or $i < n_1$ and $a_i < b_j$ then set $c_{i+j} = a_i$ and increase i by 1.
If $i \geq n_1$ or $j \leq n_2$ and $b_j \leq a_i$ then set $c_{i+j} = b_j$ and increase j by one.

Using this algorithm, we merge the two sorted halves of the array into the full sorted array.

Up to a constant factor, the recurrence relation for mergesort is $T(n) = 2T(\frac{n}{2}) + n$. As discussed in the notes on solving recurrence relations, this implies that $T(n)$ is $O(n \log n)$.

3 Counting Inversions

Problem 3.1. In the counting inversions problem, we are given a sequence $a_1, \dots, a_n$ of distinct numbers. We are then asked how many pairs $i < j \in [n]$ there are such that $i < j$ but $a_i > a_j$.

Example 3.2. For the sequence 2, 5, 3, 1, 4 there are 5 inversions. $a_1 = 2 > a_4 = 1$, $a_2 = 5 > a_3 = 3$, $a_2 = 5 > a_4 = 1$, $a_2 = 5 > a_5 = 4$, and $a_3 = 3 > a_4 = 1$.

Naïvely, solving this problem would take $O(n^2)$ time as we may have to consider every pair i, j .

Algorithm 3.3. Given an array A of n distinct elements, we can simultaneously count the number of inversions in A and sort A as follows:

Base case: If $n = 1$, the array is already sorted and there are no inversions.

Otherwise, we take the following steps:

1. Recursively apply this algorithm to elements 0 through $\lfloor \frac{n}{2} \rfloor - 1$ of A and let count_L be the number of inversions that were in these elements of A .
2. Recursively apply this algorithm to elements $\lfloor \frac{n}{2} \rfloor$ through $n - 1$ of A and let count_R be the number of inversions that were in these elements of A .
3. If A and B are two sorted arrays of lengths n_1 and n_2 then we can count the number of pairs i, j such that $a_i > b_j$ and merge these sequences into a sorted array C of length $n_1 + n_2$ as follows:

Initialization: Start with $i = 0$, $j = 0$, and $\text{count}_{\text{between}} = 0$.

Iterative step: If $j \geq n_2$ or $i < n_1$ and $a_i \leq b_j$ then set $c_{i+j} = a_i$ and increase i by 1. If $i \geq n_1$ or $j < n_2$ and $b_j < a_i$ then set $c_{i+j} = b_j$, increase $\text{count}_{\text{between}}$ by $n_1 - i$, and increase j by one.

After applying this algorithm to merge the two sorted halves of A , we have sorted A and the total number of inversions in A was $\text{count}_L + \text{count}_R + \text{count}_{\text{between}}$

To see why this merging algorithm successfully counts the number of inversions, we can think of it as follows. At each step, we have the elements in C followed by the remaining elements of A followed by the remaining elements of B . When an element of A is added to C , nothing changes. When an element b_j of B is added to C , $b_j < a_{i'}$ for all $a_{i'}$ remaining in A , so each of these remaining $a_{i'}$ was in an inversion with b_j . Shifting b_j before all of these $a_{i'}$ removes all of these inversions, reducing the number of inversions by $n_1 - i$, the number of elements remaining in A . Thus, $count_{between}$ keeps track of the number of inversions between A and B which we have removed so far. When we are finished, there will be no inversions, so $count_{between}$ will be the number of inversions which we started with between A and B .

Remark 3.4. A key advantage this algorithm has over the naïve algorithm is that it can count many inversions at once. This allows the algorithm to run in time $O(n \log n)$ rather than $O(n^2)$.

4 Closest Pair of Points

Problem 4.1. The closest pair of points problem is as follows. Given a set $P = \{p_1, \dots, p_n\} \in \mathbb{R}^2$ of n points in the plane, which two points $p_i = (x_i, y_i)$ and $p_j = (x_j, y_j)$ in P are closest to each other?

Naïvely, solving this problem would take $O(n^2)$ time as we may have to consider every pair of points p_i, p_j . Using the following divide and conquer algorithm discovered by Shamos and Hoey, this problem can be solved in $O(n \log n)$ time.

Algorithm 4.2. Given arrays X and Y of the points sorted by their x -coordinate and y -coordinate respectively, the algorithm does the following:

1. *Base case:* If $n \leq 1$ then the algorithm returns ∞ as there is no pair of points at all. If $n = 2$ then the algorithm returns p_1, p_2 as this is the only pair of points.
2. *Dividing:* We take $p_{i_{mid}} = X[\lfloor \frac{n}{2} \rfloor]$ to be the middle entry of X . We then take $P_L = \{p_i : i \text{ comes before } p_{i_{mid}} \text{ in } X\} \cup \{p_{i_{mid}}\}$ to be the points which come before $p_{i_{mid}}$ in X plus $p_{i_{mid}}$ itself and we take $P_R = \{p_i : i \text{ comes after } p_{i_{mid}} \text{ in } X\}$ to be the points which come after $p_{i_{mid}}$ in X .

After doing this, we take X_L and Y_L to be arrays containing the points in P_L sorted by their x -coordinate and y -coordinate respectively. Similarly, we take X_R and Y_R to be arrays containing the points in P_R sorted by their x -coordinate and y -coordinate respectively. Constructing X_L and X_R can be done in $O(n)$ time by going through the array X and putting the points in P_L and P_R into X_L and X_R respectively. Similarly, constructing Y_L and Y_R can be done in $O(n)$ time.

3. *Conquering:* The algorithm recursively finds the closest pair of points p_{i_L}, p_{j_L} in the left half and the closest pair of points p_{i_R}, p_{j_R} in the right half.
4. *Combining the solutions for each half into a full solution:* To find the closest pair of points overall, we also need to check whether there are any pairs of points p_i, p_j in different halves which are closer than the pairs of points we've already found. We can do this as follows:

- (a) Letting $d = \min\{d(p_{i_L}, p_{j_L}), d(p_{i_R}, p_{j_R})\}$, let $P_{mid} = \{p_i = (x_i, y_i) : |x_i - x_{mid}| < d\}$ be the subset of points which are within distance d of the dividing line $x = x_{mid}$
- (b) Take Y_{mid} to be an array of the points in P_{mid} sorted by their y -coordinate.
- (c) Consider all pairs i, j such that one of the following is true:
- i. $i = i_L$ and $j = j_L$ or $i = i_R$ and $j = j_R$.
 - ii. $p_i, p_j \in P_{mid}$ and p_j comes between 1 and 11 positions after p_i in Y_{mid} .
- Take i_{min}, j_{min} to be the pair which minimizes $d(p_i, p_j)$ among all such pairs p_i, p_j and take $d_{min} = d(p_{i_{min}}, p_{j_{min}})$.

To show that this algorithm is correct, we need to confirm that any pair of points p_i, p_j on different sides which we did not check must be at least distance $d \geq d_{min}$ from each other. We first observe that

Proposition 4.3. *If $p_i \in P_L$ and $p_j \in P_R$ then $d(p_i, p_j) \geq d$ unless $p_i, p_j \in P_{mid}$*

Proof. Observe that $x_j - x_i = (x_j - x_{mid}) + (x_{mid} - x_i)$. If $p_i \notin P_{mid}$ then $x_j - x_i \geq x_{mid} - x_i \geq d$. If $p_j \notin P_{mid}$ then $x_j - x_i \geq x_j - x_{mid} \geq d$. $\square$

Now we just need to check pairs of points in P_{mid} . For this, the key idea is that the points in P_{mid} cannot be packed too densely.

Lemma 4.4. *For each point $p_i \in P_{mid}$, there are at most 11 points $p_j \in P_{mid}$ such that $0 \leq y_j - y_i \leq d$.*

Proof. This can be shown with the following very nice idea. We partition the region $M = \{(x, y) : |x - x_{mid}| \leq d\}$ into rows of 4 boxes where each box has side length $\frac{d}{2}$. Now observe that each such box can have at most 1 point in it because otherwise there would be a pair of points in either P_L or P_R which are less than distance d apart.

For any $p_i \in P_{mid}$, we can only have that $p_j \in P_{mid}$ and $0 \leq y_j - y_i \leq d$ if p_j is in the same row of boxes as p_i or is in the one of the two rows of boxes above p_i . Thus, there are only 11 boxes which p_j could be in so there can be at most 11 such points p_j . $\square$

Combining these observations, this algorithm checks all pairs of points which could possibly be within distance d of each other and is thus correct.

5 Karatsuba algorithm for multiplication

If we want to multiply two n -digit numbers, the grade school algorithm for this takes $O(n^2)$ time. Surprisingly, it is possible to multiply numbers much faster using a divide and conquer algorithm. The first such algorithm was the Karatsuba multiplication algorithm which works as follows.

Given two n -digit numbers x and y , taking $k = \lceil \frac{n}{2} \rceil$, we can write $x = 10^k x_1 + x_2$ and $y = 10^k y_1 + y_2$ where x_1 and y_1 are $(n - k)$ -digit numbers and x_2 and y_2 are k -digit numbers. We now have that

$$xy = 10^{2k} x_1 y_1 + 10^k x_1 y_2 + 10^k x_2 y_1 + x_2 y_2$$

This allows us to compute xy by taking the sum of smaller products. Naïvely, we need to compute four products, which does not give us a faster algorithm. However, with some cleverness, we can instead only compute three products, $(x_1 + x_2)(y_1 + y_2)$, x_1y_1 , and x_2y_2 . The key observation is that $x_1y_2 + x_2y_1 = (x_1 + x_2)(y_1 + y_2) - x_1y_1 - x_2y_2$. Using this observation, we have that

$$xy = 10^{2k}x_1y_1 + 10^k((x_1 + x_2)(y_1 + y_2) - x_1y_1 - x_2y_2) + x_2y_2$$

The recurrence relation for the time taken by this algorithm is $T(n) = 3T(\frac{n}{2}) + O(n)$ which gives that $T(n)$ is $O(n^{\log_2(3)})$.

Remark 5.1. *This algorithm works for any base. In particular, this algorithm also works for binary numbers.*

6 Advanced Topic: Convolutions and the Fast Fourier Transform

6.1 The Discrete Fourier Transform

Definition 6.1. *Given $N \in \mathbb{N}$ and a function $f : [0, N-1] \cap \mathbb{Z} \rightarrow \mathbb{C}$, the discrete Fourier Transform of f is the function $\hat{f} : [0, N-1] \cap \mathbb{Z} \rightarrow \mathbb{C}$ such that*

$$\hat{f}_k := \hat{f}(k) = \sum_{j=0}^{N-1} e^{-\frac{2\pi ijk}{N}} f(j)$$

The discrete Fourier transform has several nice properties, including the following:

Theorem 6.2.

1. $\forall k \in [0, N-1] \cap \mathbb{Z}, f(k) = \sum_{j=0}^{N-1} \frac{\hat{f}_j}{N} e^{\frac{2\pi ijk}{N}}$ (Fourier decomposition)
2. $N \sum_{j=0}^{N-1} |f(j)|^2 = \sum_{j=0}^{N-1} |\hat{f}_j|^2$ (Parseval's Theorem)

The fundamental reason these properties are true is that the Fourier characters form an orthonormal basis for the vector space of functions $f : [0, N-1] \cap \mathbb{Z} \rightarrow \mathbb{C}$. We make this statement more precise below.

Definition 6.3. *We define the k th Fourier character e_k to be the function $e_k : [0, N-1] \cap \mathbb{Z} \rightarrow \mathbb{C}$ such that $e_k(j) = \frac{1}{N} e^{\frac{2\pi ijk}{N}}$.*

Definition 6.4. *Given two functions $f, g : [0, N-1] \cap \mathbb{Z} \rightarrow \mathbb{C}$, we define the inner product of f and g to be*

$$\langle f, g \rangle = N \sum_{j=0}^{N-1} f(j) \bar{g}(j)$$

where $\bar{g}$ is the complex conjugate of g .

Remark 6.5. The normalization for the inner product is strange but it is chosen to match the normalization for the discrete Fourier transform.

Proposition 6.6. The Fourier characters $\{e_k(j) : k \in [0, N-1] \cap \mathbb{Z}\}$ form an orthonormal basis for the vector space of functions $f : [0, N-1] \cap \mathbb{Z} \rightarrow \mathbb{C}$.

Proof. Observe that

$$\forall k_1, k_2 \in [0, N-1] \cap \mathbb{Z}, \langle e_{k_1}, e_{k_2} \rangle = N \sum_{j=0}^{N-1} \frac{1}{N^2} e^{\frac{2\pi i j(k_1 - k_2)}{N}}$$

which is 1 if $k_1 = k_2$ and is 0 otherwise. Thus, the Fourier characters $\{e_k(j) : k \in [0, N-1] \cap \mathbb{Z}\}$ are orthonormal. Since there are N Fourier characters and the vector space of functions $f : [0, N-1] \cap \mathbb{Z} \rightarrow \mathbb{C}$ has dimension N , the Fourier characters must form a basis as needed. $\square$

We can now prove Theorem 6.2.

Proof of Theorem 6.2. To see the first statement of Theorem 6.2, observe that if $f = \sum_{j=0}^{N-1} c_j e_j$ then $\hat{f}_k = \langle f, e_k \rangle = \langle \sum_{j=0}^{N-1} c_j e_j, e_k \rangle = c_k$. Thus, for all $j \in [0, N-1] \cap \mathbb{Z}$, $c_j = \hat{f}_j$ so $f = \sum_{j=0}^{N-1} \hat{f}_j e_j$ and thus $\forall k \in [0, N-1] \cap \mathbb{Z}$, $f(k) = \sum_{j=0}^{N-1} \frac{\hat{f}_j}{N} e^{\frac{2\pi i j k}{N}}$.

To see the second statement of Theorem 6.2, observe that since $f = \sum_{j=0}^{N-1} \hat{f}_j e_j$,

$$N \sum_{j=0}^{N-1} |f(j)|^2 = \langle f, f \rangle = \langle \sum_{j=0}^{N-1} \hat{f}_j e_j, \sum_{k=0}^{N-1} \hat{f}_k e_k \rangle = \sum_{j=0}^{N-1} \sum_{k=0}^{N-1} \hat{f}_j \overline{\hat{f}_k} \langle e_j, e_k \rangle = \sum_{j=0}^{N-1} |\hat{f}_j|^2$$

$\square$

6.2 The Fast Fourier Transform

How fast can we compute the discrete Fourier transform $\hat{f}$ of a function $f : [0, N-1] \cap \mathbb{Z} \rightarrow \mathbb{C}$? If we do it naively, it will take $\Omega(n^2)$ time. However, if N is a power of 2, we can compute $\hat{f}$ in time $O(n \log n)$ using a divide and conquer algorithm, the fast Fourier transform. The algorithm works as follows:

Algorithm 6.7. Given a function $f : [0, N-1] \cap \mathbb{Z} \rightarrow \mathbb{C}$ where N is a power of 2,

1. Take $f_{\text{even}} : [0, \frac{N}{2}-1] \cap \mathbb{Z} \rightarrow \mathbb{C}$ to be the function such that $f_{\text{even}}(k) = f(2k)$ and take $f_{\text{odd}} : [0, \frac{N}{2}-1] \cap \mathbb{Z} \rightarrow \mathbb{C}$ to be the function such that $f_{\text{odd}}(k) = f(2k+1)$
2. Compute the discrete Fourier transforms of $\hat{f}_{\text{even}}$ and $\hat{f}_{\text{odd}}$.
3. Observe that

(a) For all $k \in [0, \frac{N}{2} - 1] \cap \mathbb{Z}$,

$$\begin{aligned}
\hat{f}_k &= \sum_{j=0}^{N-1} e^{\frac{-2\pi i k j}{N}} f(j) \\
&= \sum_{j=0}^{\frac{N}{2}-1} e^{\frac{-2\pi i k (2j)}{N}} f(2j) + \sum_{j=0}^{\frac{N}{2}-1} e^{\frac{-2\pi i k (2j+1)}{N}} f(2j+1) \\
&= \sum_{j=0}^{\frac{N}{2}-1} e^{\frac{-2\pi i k j}{\frac{N}{2}}} f_{\text{even}}(j) + e^{\frac{-2\pi i k}{N}} \sum_{j=0}^{\frac{N}{2}-1} e^{\frac{-2\pi i k j}{\frac{N}{2}}} f_{\text{odd}}(j) \\
&= \hat{f}_{\text{even}}(k) + e^{\frac{-2\pi i k}{N}} \hat{f}_{\text{odd}}(k)
\end{aligned}$$

(b) For all $k \in [0, \frac{N}{2} - 1] \cap \mathbb{Z}$,

$$\begin{aligned}
\hat{f}_{\frac{N}{2}+k} &= \sum_{j=0}^{N-1} e^{\frac{-2\pi i (\frac{N}{2}+k) j}{N}} f(j) \\
&= \sum_{j=0}^{\frac{N}{2}-1} e^{\frac{-2\pi i (\frac{N}{2}+k) (2j)}{N}} f(2j) + \sum_{j=0}^{\frac{N}{2}-1} e^{\frac{-2\pi i (\frac{N}{2}+k) (2j+1)}{N}} f(2j+1) \\
&= \sum_{j=0}^{\frac{N}{2}-1} e^{\frac{-2\pi i k j}{\frac{N}{2}}} f_{\text{even}}(j) + e^{\frac{-2\pi i (\frac{N}{2}+k)}{N}} \sum_{j=0}^{\frac{N}{2}-1} e^{\frac{-2\pi i k j}{\frac{N}{2}}} f_{\text{odd}}(j) \\
&= \hat{f}_{\text{even}}(k) - e^{\frac{-2\pi i k}{N}} \hat{f}_{\text{odd}}(k)
\end{aligned}$$

6.3 Convolutions

Definition 6.8. Given two functions $f, g : [0, N-1] \cap \mathbb{Z} \rightarrow \mathbb{C}$, we define the convolution $f * g : [0, N-1] \cap \mathbb{Z} \rightarrow \mathbb{C}$ to be the function such that $(f * g)(k) = \sum_{j_1, j_2: j_1+j_2=k \bmod N} f(j_1)g(j_2)$

Theorem 6.9. For any two functions $f, g : [0, N-1] \cap \mathbb{Z} \rightarrow \mathbb{C}$ and any $k \in [0, N-1] \cap \mathbb{Z}$, $(f \hat{*} g)_k = \hat{f}_k \hat{g}_k$.

Proof.

$$\begin{aligned}
(f \hat{*} g)_k &= \sum_{j=0}^{N-1} e^{\frac{-2\pi i k j}{N}} (f * g)(j) \\
&= \sum_{j=0}^{N-1} \sum_{j'=0}^{N-1} e^{\frac{-2\pi i k (j' + (j-j'))}{N}} f(j') g((j-j') \bmod N) \\
&= \sum_{j=0}^{N-1} \sum_{j'=0}^{N-1} e^{\frac{-2\pi i k (j'+j)}{N}} f(j') g(j) \\
&= \left(\sum_{j'=0}^{N-1} e^{\frac{-2\pi i k j'}{N}} f(j') \right) \left(\sum_{j=0}^{N-1} e^{\frac{-2\pi i k j}{N}} g(j) \right) = \hat{f}_k \hat{g}_k
\end{aligned}$$

□

Thus, if N is a power of 2 we can evaluate the convolution $f * g$ on all inputs in $O(N \log N)$ time by finding the discrete Fourier transforms $\hat{f}$ and $\hat{g}$ of f and g , taking $(f * g) = \hat{f} \hat{g}$, and then inverting the discrete Fourier transform on $(\hat{f} \hat{g})$ to find $f * g$.